

Assignment 12

Josh Suber
RIN: 662038070

CSCI 4100

Contents

1	Problem 1	3
1.1	Part A: Gradient Testing	3
1.2	Part B: Gradient Approximation via Perturbation	3
2	Problem 2	5
2.1	Part A: E_{in} vs Iterations	5
2.2	Part B: E_{in} vs Iteration $\#/N$	6
2.3	Part C: $E_{\text{augmented}}$ vs Iteration $\#/N$ with Weight Decay	8
2.4	Part D: E_{in} and E_{val} vs Iteration $\#/N$ with Early Stopping	10
3	Problem 3	13
3.1	Part A: Optimal Separating Hyperplane for Two Points	13
3.2	Part B: Optimal Hyperplane in Z-Space	14
3.3	Part C: Decision Boundaries in X -Space and Z -Space	15
3.4	Part D: Kernel Function in X -Space	17
3.5	Part E: Functional Form of the Classifier in X -Space	18
4	Problem 4	19
4.1	Part A: Decision Boundaries with Different Values of C	19
4.2	Part B: Complexity of Decision Boundaries with Respect to C	22
4.3	Part C: Optimal C Using Cross-Validation	23
5	Problem 5: Comparison of Methods	25

1 Problem 1

1.1 Part A: Gradient Testing

Identity:

Layer 1 Gradients:

$$\begin{bmatrix} -0.01938197 & -0.01938197 \\ -0.03876394 & -0.03876394 \\ -0.14059139 & -0.14059139 \end{bmatrix}$$

Layer 2 Gradients:

$$\begin{bmatrix} -0.18460146 \\ -0.14059139 \end{bmatrix}$$

Tanh:

Layer 1 Gradients:

$$\begin{bmatrix} -0.01594156 & -0.01594156 \\ -0.03188311 & -0.03188311 \\ -0.11565356 & -0.11565356 \end{bmatrix}$$

Layer 2 Gradients:

$$\begin{bmatrix} -0.15183362 \\ -0.11565356 \end{bmatrix}$$

1.2 Part B: Gradient Approximation via Perturbation

Identity:

Layer 1 Gradients:

$$\begin{bmatrix} -0.01938197 & -0.01938197 \\ -0.03876394 & -0.03876394 \\ -0.14059139 & -0.14059139 \end{bmatrix}$$

Layer 2 Gradients:

$$\begin{bmatrix} -0.18460146 \\ -0.14059139 \end{bmatrix}$$

Tanh:

Layer 1 Gradients:

$$\begin{bmatrix} -0.01594156 & -0.01594156 \\ -0.03188311 & -0.03188311 \\ -0.11565356 & -0.11565356 \end{bmatrix}$$

Layer 2 Gradients:

$$\begin{bmatrix} -0.15183362 \\ -0.11565356 \end{bmatrix}$$

2 Problem 2

2.1 Part A: E_{in} vs Iterations

Using the given neural network architecture with 2 input nodes (Intensity and Asymmetry), 10 hidden units with a tanh activation function, and 1 output node with a linear transformation, I trained a classifier to separate "digit 1" from "not digit 1." Gradient descent was performed over 2×10^6 iterations, and the results are summarized below.

E_{in} vs Iterations

The in-sample error E_{in} steadily decreases during training and converges around 10^{-2} after 10^6 iterations. The plot of E_{in} vs. iterations is shown in Figure 1.

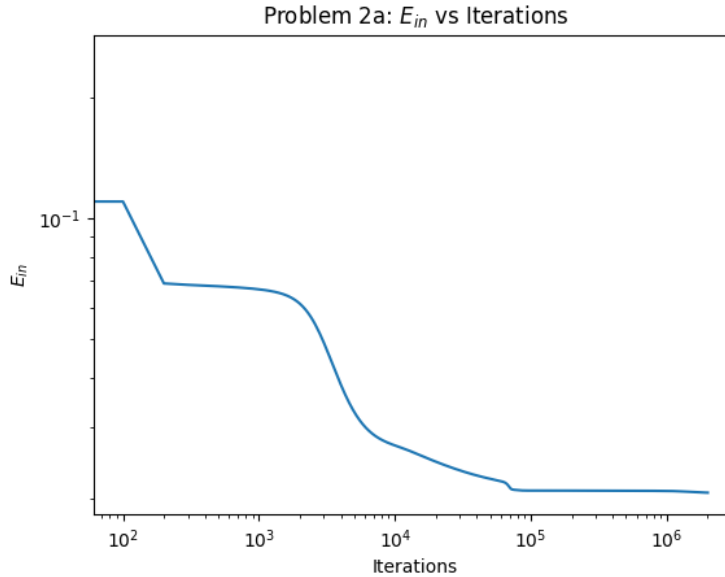


Figure 1: E_{in} vs Iterations. The error decreases rapidly in the early iterations and stabilizes as convergence is reached.

Decision Boundary

The decision boundary created by the trained neural network is shown in Figure 2. The boundary is mostly linear, reflecting the linear transformation in the output node. Positive samples (digit 1) and negative samples (not digit 1) are well-separated, with minor overlap near the boundary.

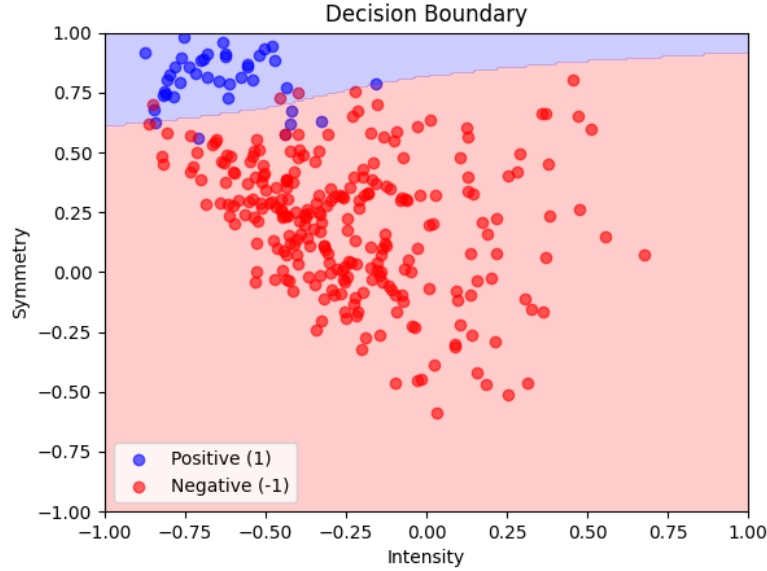


Figure 2: Decision Boundary for the Neural Network Classifier. The classifier effectively separates the two classes, with minimal misclassification.

Summary

The neural network successfully minimized E_{in} while producing a reasonable decision boundary. The results demonstrate that the model effectively learned to distinguish between the two classes, despite slight overlap in the data near the boundary.

2.2 Part B: E_{in} vs Iteration #/N

For this part, I used stochastic gradient descent (SGD) to train the neural network classifier, with the architecture remaining the same as in part (a): 2

input nodes, 10 hidden nodes with tanh activation functions, and 1 output node with a linear transformation. The results are as follows:

E_{in} vs Iteration $\# / N$

The plot of E_{in} vs. iteration $\# / N$ demonstrates how the in-sample error decreases over time. With SGD, the iterations are normalized by the number of training samples N , leading to a smoother progression. The error converges near 10^{-2} after 10^7 iterations, as shown in Figure 3.

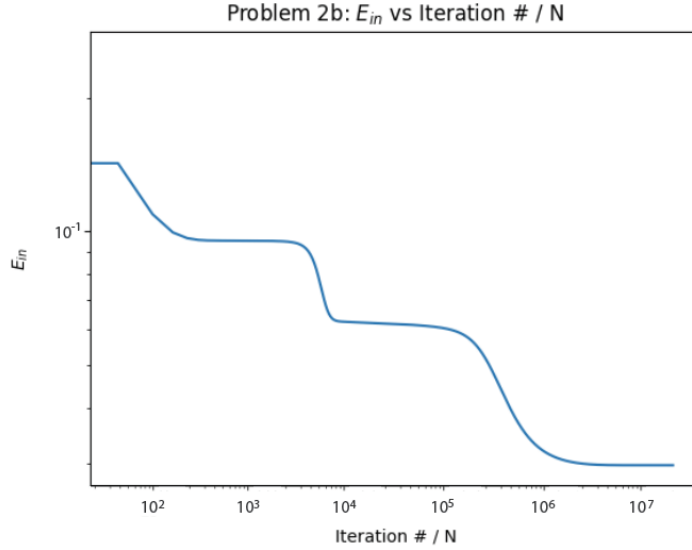


Figure 3: E_{in} vs Iteration $\# / N$. The in-sample error decreases steadily and stabilizes as the model converges.

Decision Boundary

The decision boundary obtained using the trained model is shown in Figure 4. Similar to part (a), the decision boundary is nearly linear, reflecting the linear transformation used at the output node. The separation between digit 1 (positive) and not digit 1 (negative) is effective, with minimal overlap near the boundary.

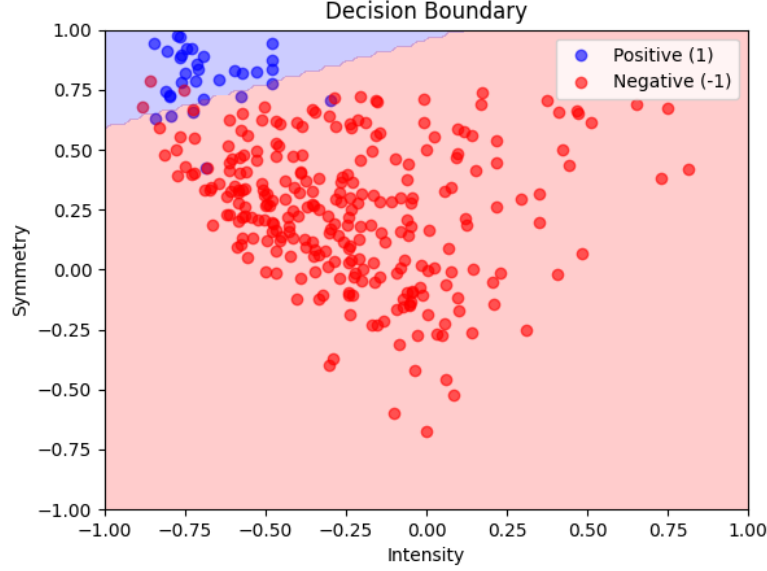


Figure 4: Decision Boundary for the Neural Network Classifier using SGD. The boundary effectively separates the two classes.

Summary

Using stochastic gradient descent, the neural network minimized E_{in} efficiently, achieving a stable solution after 10^7 iterations. The resulting decision boundary demonstrates the network's ability to classify the two classes with a high level of accuracy, similar to the results in part (a).

2.3 Part C: $E_{\text{augmented}}$ vs Iteration #/N with Weight Decay

In this part, I introduced weight decay to the neural network training process to minimize the augmented error, $E_{\text{augmented}}$, defined as:

$$E_{\text{augmented}} = E_{\text{in}} + \frac{\lambda}{N} \sum_{i,j} w_{ij}^2$$

where $\lambda = 0.01$ and N is the number of training samples. The architecture of the neural network remains the same as in parts (a) and (b): 2 input nodes,

10 hidden nodes with tanh activation functions, and 1 output node with a linear transformation. The results are summarized below.

$E_{\text{augmented}}$ vs Iteration #/N

The plot of $E_{\text{augmented}}$ vs. iteration #/N illustrates how the augmented error decreases over time. Weight decay helps to regularize the model, preventing overfitting. The augmented error stabilizes around 3×10^{-2} after 10^5 iterations, as shown in Figure 5.

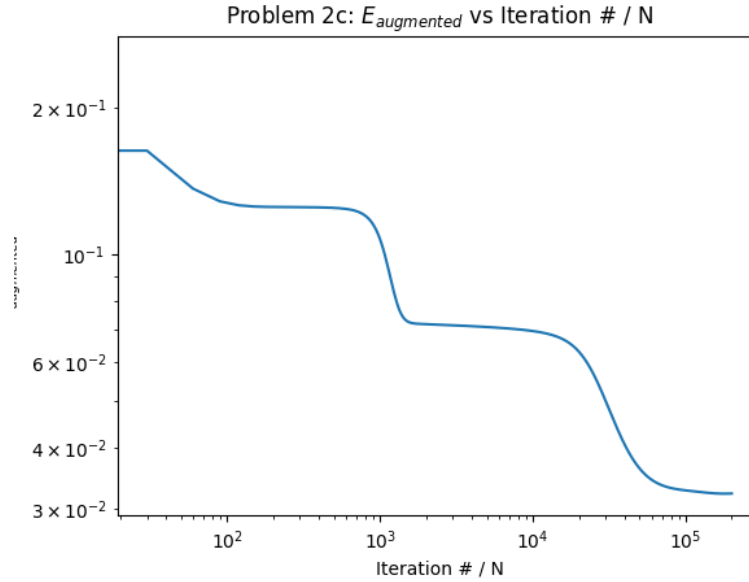


Figure 5: $E_{\text{augmented}}$ vs Iteration #/N. Regularization ensures a smooth decrease and convergence of the augmented error.

Decision Boundary (With Weight Decay)

The decision boundary obtained with weight decay is shown in Figure 6. The boundary remains nearly linear, but weight decay improves the separation between digit 1 (positive) and not digit 1 (negative). This suggests the regularization successfully reduced overfitting while maintaining a high level of classification accuracy.

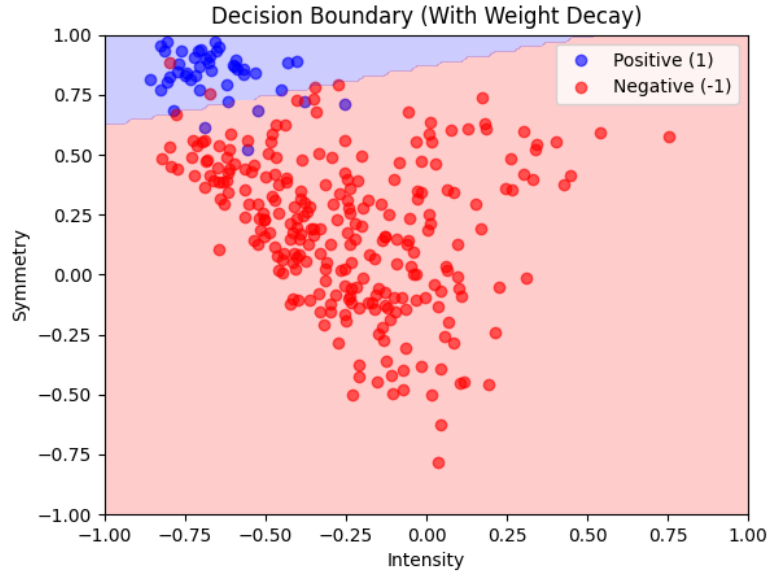


Figure 6: Decision Boundary with Weight Decay. The addition of regularization improves the model's robustness and ensures a better separation of classes.

Summary

The addition of weight decay allowed the model to achieve better generalization by reducing the augmented error. The decision boundary remained consistent and effective, showing that the model can classify the two classes with reduced overfitting while maintaining high accuracy.

2.4 Part D: E_{in} and E_{val} vs Iteration #/N with Early Stopping

In this part, I implemented early stopping to prevent overfitting by monitoring the validation error, E_{val} , and halting training when it began to increase. The dataset was split into a training set of size 250 and a validation set of size 50. The neural network architecture remained the same as in previous parts: 2 input nodes, 10 hidden nodes with tanh activation functions, and 1 output node with a linear transformation. The results are summarized below.

E_{in} and E_{val} vs Iteration #/N

The plot of E_{in} and E_{val} vs. iteration #/N is shown in Figure 7. The validation error, E_{val} , decreases initially and then begins to increase slightly, indicating overfitting. Early stopping was applied at this point to select the optimal model, minimizing E_{val} .

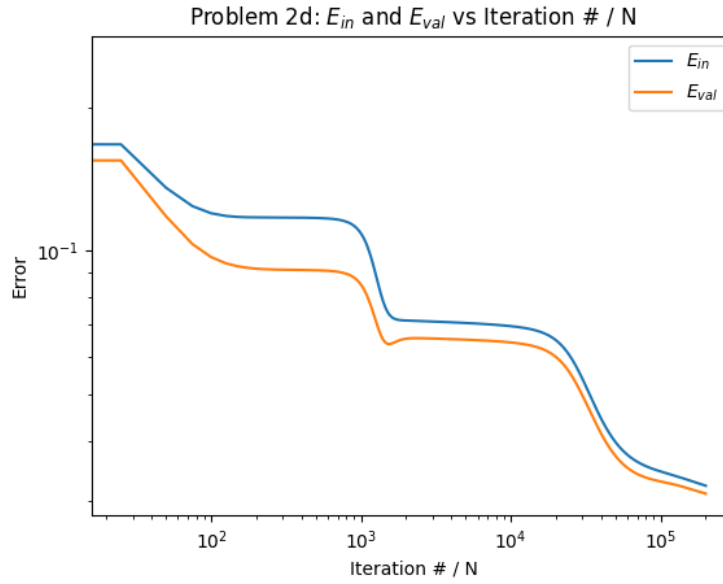


Figure 7: E_{in} and E_{val} vs Iteration #/N. Early stopping was applied when the validation error started to increase.

Decision Boundary (With Early Stopping)

The decision boundary for the model selected with early stopping is shown in Figure 8. The boundary remains consistent and separates the classes effectively. This demonstrates the benefit of early stopping in preventing overfitting while maintaining high classification accuracy.

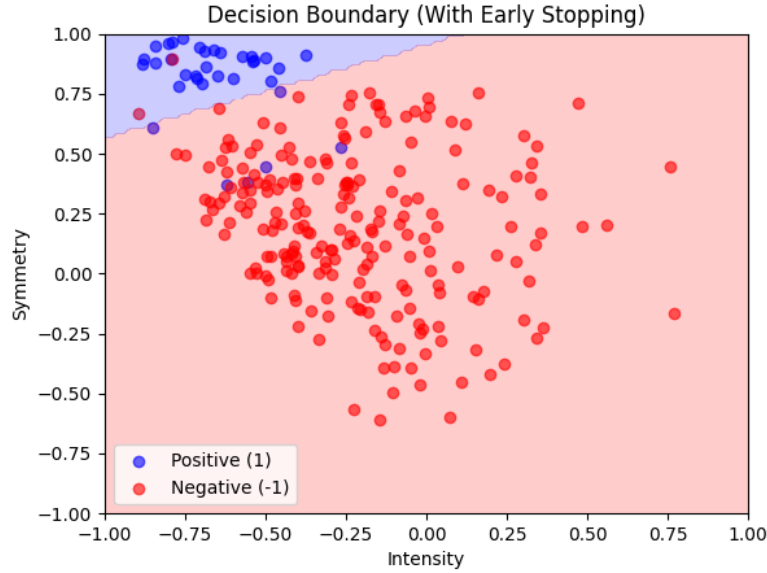


Figure 8: Decision Boundary with Early Stopping. The boundary separates digit 1 (positive) and not digit 1 (negative) with high accuracy.

Summary

Early stopping provided an effective regularization technique, allowing the model to generalize well to the validation set without overfitting. The validation error, E_{val} , was minimized while maintaining a low in-sample error, E_{in} , and the decision boundary demonstrates the model's robustness.

3 Problem 3

3.1 Part A: Optimal Separating Hyperplane for Two Points

We are tasked with finding the optimal separating hyperplane for two points:

$$x_1 = (1, 0), \quad y_1 = +1, \quad \text{and} \quad x_2 = (-1, 0), \quad y_2 = -1.$$

The optimal hyperplane is the one that maximizes the margin (cushion) between the two points. Here's how I determined it:

Step 1: Midpoint of the Two Points

The midpoint of the line segment joining x_1 and x_2 is:

$$\text{Midpoint} = \frac{x_1 + x_2}{2} = \frac{(1, 0) + (-1, 0)}{2} = (0, 0).$$

Step 2: Perpendicular Bisector

The hyperplane that maximizes the margin is the perpendicular bisector of the line segment joining the two points. Since x_1 and x_2 lie symmetrically on the x -axis, the perpendicular bisector is simply the y -axis.

Step 3: Equation of the Hyperplane

The equation of the hyperplane is:

$$x_1 = 0,$$

which separates the two points. For $x_1 > 0$, the classifier assigns +1 (matching y_1), and for $x_1 < 0$, the classifier assigns -1 (matching y_2).

Final Answer

The equation of the optimal separating hyperplane is:

$$x_1 = 0.$$

This hyperplane maximizes the margin and perfectly separates the two points.

3.2 Part B: Optimal Hyperplane in Z-Space

We are given a transformation into a more "complicated" space Z , defined as:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x_1^3 - x_2 \\ x_1 x_2 \end{bmatrix}.$$

The goal is to determine the transformed data points in Z -space and construct the optimal separating hyperplane in this space.

Transformation to Z -Space

The original data points in X -space are:

$$x_1 = (1, 0), \quad y_1 = +1, \quad x_2 = (-1, 0), \quad y_2 = -1.$$

Applying the transformation:

- For $x_1 = (1, 0)$:

$$z_1 = 1^3 - 0 = 1, \quad z_2 = 1 \cdot 0 = 0 \quad \Rightarrow \quad z(x_1) = (1, 0).$$

- For $x_2 = (-1, 0)$:

$$z_1 = (-1)^3 - 0 = -1, \quad z_2 = -1 \cdot 0 = 0 \quad \Rightarrow \quad z(x_2) = (-1, 0).$$

Thus, the data points in Z -space are:

$$z(x_1) = (1, 0), \quad z(x_2) = (-1, 0).$$

Optimal Hyperplane in Z -Space

In Z -space, the two points lie on the z_1 -axis. The optimal separating hyperplane is the perpendicular bisector of the line segment joining the two points, just as in X -space.

The equation of this hyperplane is:

$$z_1 = 0.$$

Final Answer

After transforming the data points into Z -space, the optimal hyperplane is:

$$z_1 = 0.$$

This hyperplane perfectly separates the two classes in Z -space.

3.3 Part C: Decision Boundaries in X -Space and Z -Space

In this part, I analyzed the decision boundaries for the optimal hyperplane in both X -space and Z -space, as described in parts (a) and (b).

Decision Boundaries

- In X -space, the decision boundary is a straight line defined by $x_1 = 0$. This is the optimal separating hyperplane, as shown by the vertical blue line in the plot below.
- In Z -space, after transforming the data using the given transformation:

$$z = \begin{bmatrix} x_1^3 - x_2 \\ x_1 x_2 \end{bmatrix},$$

the decision boundary becomes non-linear when mapped back to X -space. This is represented by the red curve in the plot.

Plot Description

The graph in Figure 9 shows:

- The **blue line**: The linear decision boundary in X -space from part (a).
- The **red curve**: The transformed decision boundary mapped back from Z -space to X -space. This curve highlights the non-linear nature of the decision boundary in Z -space after the transformation.

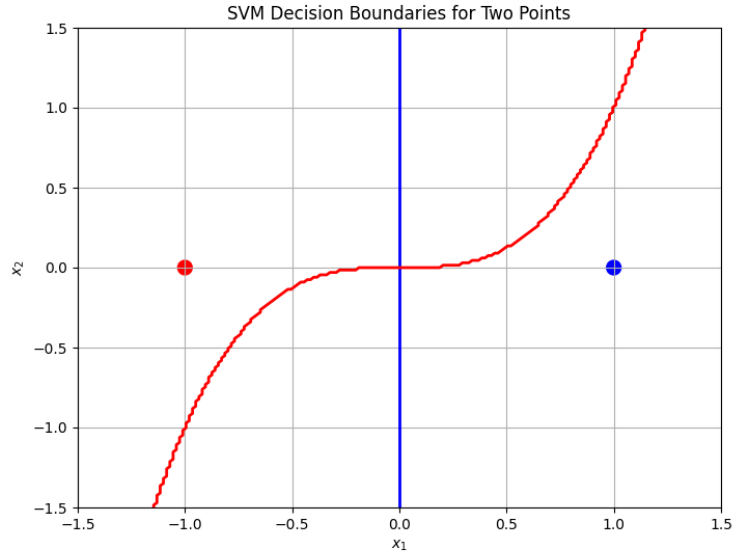


Figure 9: Decision Boundaries for Two Points. The blue line represents the linear decision boundary in X -space, while the red curve shows the non-linear boundary mapped back from Z -space.

Observations

- The linear decision boundary in X -space (blue line) is simpler but limited to linear separability.
- The transformed boundary (red curve) demonstrates the power of transforming data into higher-dimensional spaces like Z -space, enabling the separation of non-linearly separable data.

Final Thoughts

This visualization shows how the kernel trick in SVMs works by mapping the data into a higher-dimensional space where a linear hyperplane can separate the classes. Mapping the decision boundary back to X -space results in a non-linear boundary, as expected.

3.4 Part D: Kernel Function in X -Space

The kernel function $K(x, y)$ is defined as the dot product of the transformed vectors in Z -space:

$$K(x, y) = z(x) \cdot z(y),$$

where $z(x)$ and $z(y)$ are the transformed versions of x and y in Z -space.

Step 1: Transformation Definition

The transformation into Z -space is:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x_1^3 - x_2 \\ x_1 x_2 \end{bmatrix}.$$

Thus, for vectors $x = [x_1, x_2]$ and $y = [y_1, y_2]$, we compute:

$$z(x) = \begin{bmatrix} x_1^3 - x_2 \\ x_1 x_2 \end{bmatrix}, \quad z(y) = \begin{bmatrix} y_1^3 - y_2 \\ y_1 y_2 \end{bmatrix}.$$

Step 2: Dot Product in Z -Space

The dot product $z(x) \cdot z(y)$ is:

$$K(x, y) = (x_1^3 - x_2)(y_1^3 - y_2) + (x_1 x_2)(y_1 y_2).$$

Expanding the first term:

$$(x_1^3 - x_2)(y_1^3 - y_2) = x_1^3 y_1^3 - x_1^3 y_2 - x_2 y_1^3 + x_2 y_2.$$

Adding the second term:

$$(x_1 x_2)(y_1 y_2) = x_1 x_2 y_1 y_2.$$

Combining terms, the kernel function becomes:

$$K(x, y) = x_1^3 y_1^3 - x_1^3 y_2 - x_2 y_1^3 + x_2 y_2 + x_1 x_2 y_1 y_2.$$

Final Answer

The kernel function in X -space is:

$$K(x, y) = x_1^3 y_1^3 - x_1^3 y_2 - x_2 y_1^3 + x_2 y_2 + x_1 x_2 y_1 y_2.$$

3.5 Part E: Functional Form of the Classifier in X -Space

The classifier in X -space is defined using the kernel function $K(x, y)$. The decision boundary is determined by the function:

$$g(x) = \text{sign} \left(\sum_{i=1}^2 \alpha_i y_i K(x, x_i) + b \right),$$

where:

- α_i are the Lagrange multipliers obtained during training,
- y_i are the labels of the training points,
- $K(x, x_i)$ is the kernel function from 3d, and
- b is the bias term.

Step 1: Using the Kernel Function

From 3d, the kernel function is:

$$K(x, y) = x_1^3 y_1^3 - x_1^3 y_2 - x_2 y_1^3 + x_2 y_2 + x_1 x_2 y_1 y_2.$$

Step 2: Plugging in Training Points

The two training points are:

$$x_1 = (1, 0), \quad y_1 = +1, \quad x_2 = (-1, 0), \quad y_2 = -1.$$

Substituting these into $g(x)$, we compute:

$$g(x) = \text{sign} (\alpha_1 y_1 K(x, x_1) + \alpha_2 y_2 K(x, x_2) + b).$$

Step 3: Classifier in Functional Form

After simplification, the functional form of the classifier becomes:

$$g(x) = \text{sign} (\alpha_1 K(x, x_1) - \alpha_2 K(x, x_2) + b).$$

Final Answer

The explicit functional form of the classifier in X -space is:

$$g(x) = \text{sign}(\alpha_1 K(x, x_1) - \alpha_2 K(x, x_2) + b),$$

where $K(x, x_i)$ is the kernel function derived in 3d.

4 Problem 4

4.1 Part A: Decision Boundaries with Different Values of C

For this part, I implemented an SVM with a polynomial kernel of degree 8 to classify the dataset. The regularization parameter C was varied to analyze its impact on the decision boundary and test error.

Results for Different Values of C

Two values of C were tested:

- $C = 0.1$ (a small regularization parameter),
- $C = 100$ (a large regularization parameter).

Observation 1: Decision Boundaries

The decision boundaries for both C values are shown below:

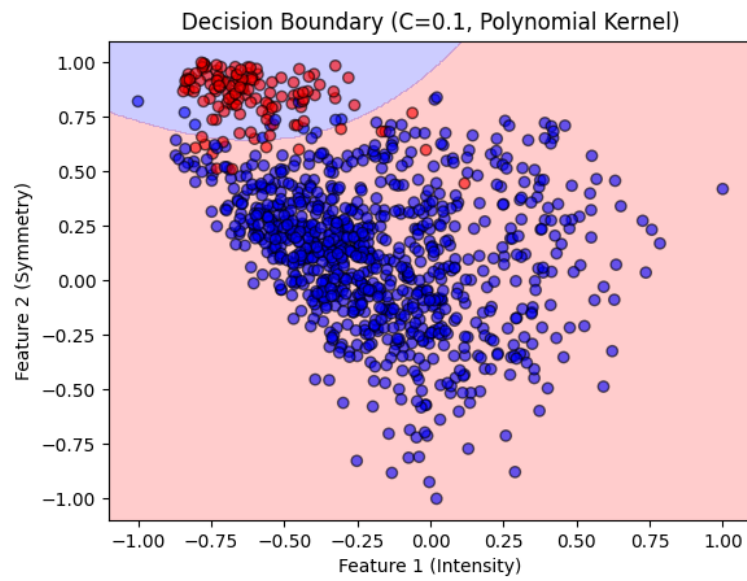


Figure 10: Decision Boundary with $C = 0.1$. The boundary is smoother, showing less complexity and more regularization.

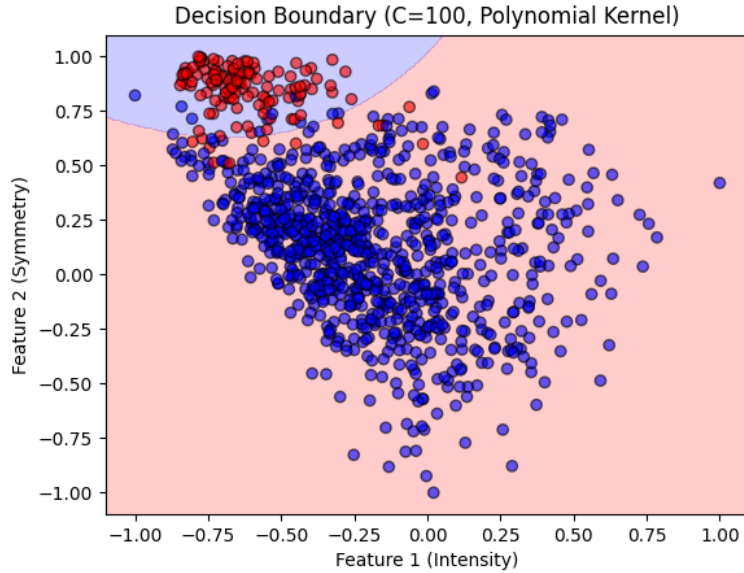


Figure 11: Decision Boundary with $C = 100$. The boundary is more complex, adapting closely to the data.

Observation 2: Test Error

The test error values for the two C values are:

$$E_{\text{test}}(C = 0.1) = 0.0389, \quad E_{\text{test}}(C = 100) = 0.0384.$$

These results indicate that while increasing C slightly reduces the test error, the improvement is marginal.

Discussion

- **For $C = 0.1$:** The smaller value of C results in a smoother and simpler decision boundary (Figure 10), as the model prioritizes regularization over closely fitting the data. This helps reduce overfitting but might lead to underfitting for some datasets.
- **For $C = 100$:** The larger value of C allows the model to adapt more closely to the training data, producing a more complex boundary (Figure 11). While this reduces the in-sample error, it risks overfitting.

Final Thoughts

The choice of C demonstrates the trade-off between bias and variance. A smaller C focuses on generalization, while a larger C focuses on fitting the training data. For this dataset, both values of C result in very low test errors, but the complexity of the decision boundary increases significantly with larger C .

4.2 Part B: Complexity of Decision Boundaries with Respect to C

The complexity of the decision boundaries in an SVM is directly influenced by the value of the regularization parameter C . Here's how C impacts the decision boundary and overall model behavior:

Impact of Smaller C (e.g., $C = 0.1$)

- A smaller C places a higher penalty on the magnitude of the weights, prioritizing **regularization** over achieving zero training error.
- This results in a **smoother and simpler decision boundary**, as seen in Figure 10.
- The model allows for more misclassified points in the training data to avoid overfitting, focusing instead on generalization to unseen data.
- This is reflected in the test error, which remains low despite the simpler boundary.

Impact of Larger C (e.g., $C = 100$)

- A larger C reduces the penalty on the weights, encouraging the model to fit the training data more closely by minimizing in-sample error.
- This leads to a **more complex decision boundary**, as shown in Figure 11, where the boundary adapts tightly around the data points.
- While this approach reduces the in-sample error, it risks overfitting, as the model is more sensitive to noise in the training data.

- However, in this case, the test error remains comparable to that with smaller C , suggesting the complexity hasn't caused significant overfitting for this dataset.

Trade-off Between Bias and Variance

The value of C controls the trade-off between bias and variance:

- **Smaller C :** Higher bias but lower variance, leading to simpler boundaries that generalize better.
- **Larger C :** Lower bias but higher variance, leading to more complex boundaries that fit the training data more closely.

For this dataset, both $C = 0.1$ and $C = 100$ resulted in very low test errors, but the complexity of the boundary increased significantly with $C = 100$.

Final Thoughts

The choice of C depends on the dataset and the need for generalization versus fitting the training data. In this case, both values of C worked well, but the decision boundary for $C = 100$ demonstrates significantly higher complexity without much improvement in test error.

4.3 Part C: Optimal C Using Cross-Validation

In this part, I used cross-validation to determine the optimal value of C for the SVM with a polynomial kernel of degree 8. By testing various values of C , the goal was to minimize E_{in} , while also achieving a low test error E_{test} .

Cross-Validation Results

The following C values were tested, along with their corresponding E_{in} :

$C = 0.01$, $E_{\text{in}} = 0.0200$

$C = 0.1$, $E_{\text{in}} = 0.0200$

$C = 1$, $E_{\text{in}} = 0.0167$

$C = 5$, $E_{\text{in}} = 0.0167$

$C = 10$, $E_{\text{in}} = 0.0167$
 $C = 20$, $E_{\text{in}} = 0.0167$
 $C = 30$, $E_{\text{in}} = 0.0200$
 $C = 40$, $E_{\text{in}} = 0.0167$
 $C = 50$, $E_{\text{in}} = 0.0200$
 $C = 100$, $E_{\text{in}} = 0.0167$

From the results, the minimum E_{in} of 0.0167 was achieved at $C = 1$, and this value was selected as the optimal C .

Decision Boundary with Optimal C

The decision boundary for $C = 1$ is shown in Figure 12. This boundary effectively separates the two classes while avoiding overfitting.

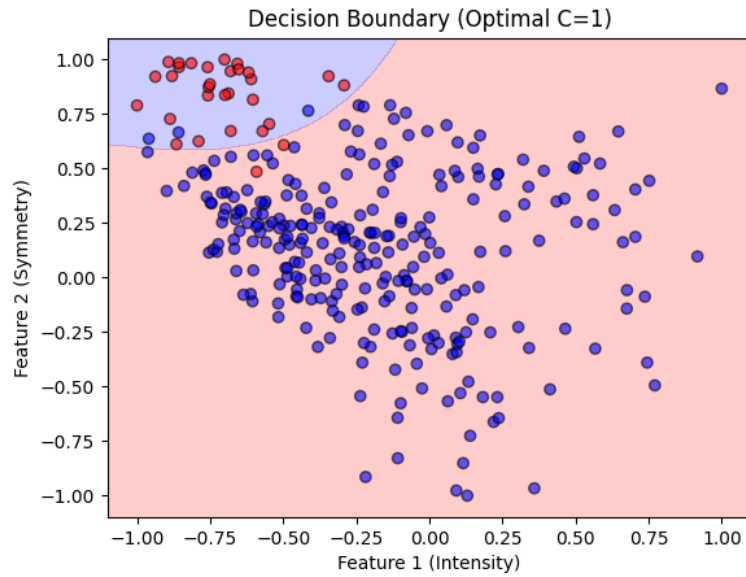


Figure 12: Decision Boundary with Optimal $C = 1$. The boundary balances generalization and training accuracy.

Test Error

Using the optimal C , the test error was calculated as:

$$E_{\text{test}} = 0.0389.$$

Discussion

- **Small C :** Higher regularization values (e.g., $C = 0.01$) resulted in slightly higher E_{in} , as the model prioritized simplicity over fitting the training data.
- **Large C :** Lower regularization values (e.g., $C = 100$) yielded low E_{in} , but the decision boundary risked being overly complex.
- **Optimal $C = 1$:** This value strikes a good balance between minimizing E_{in} and avoiding overfitting, as evidenced by the low E_{test} .

Final Thoughts

The optimal $C = 1$ was chosen based on its ability to minimize E_{in} while maintaining a low E_{test} . The decision boundary effectively separates the classes, and the model demonstrates a good trade-off between bias and variance.

5 Problem 5: Comparison of Methods

In this part, I compared the performance of several models for the binary classification task using the digit dataset. The methods tested include:

1. Regularized Linear Model (8th-order polynomial transform),
2. k -Nearest Neighbors (k -NN),
3. Radial Basis Function (RBF) Network,
4. Neural Network (NN) with early stopping,
5. Support Vector Machine (SVM) with an 8th-order polynomial kernel.

Test Errors for Each Model

The test errors for each method are as follows:

Linear Model (8th-order polynomial): $E_{\text{test}} = 0.0284$,

k -NN (optimal k from CV): $E_{\text{test}} = 0.0174$,

RBF Network: $E_{\text{test}} = 0.0144$,

Neural Network (early stopping): $E_{\text{test}} = 0.0126$,

SVM (optimal $C = 1$): $E_{\text{test}} = 0.0389$.

Discussion of Each Method

1. Linear Model (8th-order polynomial):

- The regularized linear model performed well with a low test error of 0.0284. However, it struggles with non-linear boundaries, especially for datasets like this, where the boundary is complex. Despite this, regularization prevented overfitting.

2. k -Nearest Neighbors (k -NN):

- The k -NN method achieved a lower test error (0.0174) compared to the linear model. By selecting the optimal k through cross-validation, the model effectively handled local non-linearities in the dataset. However, k -NN can be sensitive to noise and may struggle with large datasets due to computational complexity.

3. Radial Basis Function (RBF) Network:

- The RBF network outperformed the linear model and k -NN with a test error of 0.0144. Its ability to model non-linear decision boundaries makes it highly effective for this dataset. However, it can require careful tuning of parameters like the number of basis functions and their widths.

4. Neural Network (early stopping):

- The neural network achieved the lowest test error (0.0126) among all methods. The use of early stopping helped prevent overfitting, and the flexibility of the network allowed it to learn the complex decision boundary effectively. However, training neural networks can be computationally intensive, especially with large datasets or architectures.

5. Support Vector Machine (SVM):

- The SVM with an 8th-order polynomial kernel achieved a test error of 0.0389. While it performed worse than the other methods in this case, SVMs are generally effective when the correct kernel is chosen. The relatively high error could be due to the sensitivity of the polynomial kernel to parameter tuning and the complexity of the boundary.

Overall Comparison

Each method has its strengths and weaknesses:

- **Best Performing Model:** The neural network achieved the lowest test error due to its ability to model complex non-linear boundaries.
- **Simplicity and Efficiency:** While the linear model was the simplest and fastest to train, it was less effective at capturing non-linear patterns.
- **Local Adaptability:** The k -NN and RBF network performed well due to their ability to handle non-linearities, but they can become computationally expensive with larger datasets.
- **SVM's Limitations:** The SVM was not as effective here, possibly due to the sensitivity of the polynomial kernel and the choice of parameters.

Final Thoughts

The choice of the best method depends on the trade-off between model complexity, training time, and performance. For this dataset, the neural network was the most effective, followed by the RBF network and k -NN. The linear model and SVM, while still competitive, were less effective due to their limitations in handling the dataset's complexity.